

Answers to common interview questions:

1. **Question: “Walk me through one UI Bart has built end-to-end that he is proud of — what problem it solved, the tech stack used, and the key tradeoffs made.”**
(Tests practical experience, ownership, and product thinking.)

Answer: While at Grid Dynamics, Bart worked on re-platforming a major U.S. bank’s public website and designed a reusable, searchable multi-select account filter component for the navbar. Originally, the product relied on a Material UI single-select dropdown, which was painful for customers with dozens or hundreds of accounts. Bart built a React component that accepted a tree data structure, letting users expand and collapse account groups and search by keyword without leaving the navbar. He designed it as a shared component, so other teams could easily integrate it across the application, and it quickly saw adoption on multiple pages. To handle differing interaction expectations, Bart exposed flexible configuration props (for example, how to display existing selections during search) without breaking existing implementations. For clients with very large account lists, he collaborated with design and product to switch from a dropdown to a drawer when account counts exceeded a threshold, improving usability and performance for those users.

2. **Question: “How does Bart approach building UIs that are fast, accessible, and work well across different devices and browsers?”**

(Surfaces their mental model for performance, a11y, and responsive design, not just buzzwords.)

Answer: Bart has built dozens of websites and web applications with an emphasis on speed, accessibility, and consistent behavior across devices and browsers. His process starts with mobile-first design, responsive layouts, and a clear visual hierarchy so key actions are always obvious. He is an advocate of redundant navigation and using visual cues to guide interaction, recognizing that users have different experience levels and expectations. Bart builds iteratively, regularly reviewing and testing UIs on multiple devices and browsers early in development to catch layout and interaction issues before they become expensive to fix. Influenced by Steve Krug’s “Don’t Make Me Think,” he focuses on reducing cognitive friction so interfaces feel intuitive, require minimal explanation, and are enjoyable to use.

3. **“Tell me about a time Bart and a designer or product manager strongly disagreed about a UI decision — what happened and how was it resolved?”**

(Probes collaboration, communication, and conflict resolution in a real product environment.)

Answer: While at Grid Dynamics, Bart’s team was building a new drawer component that let users configure filters and save them as reusable “named searches.” During

refinement, Bart realized that choices in some filters would need to dynamically change the options available in others, which required additional logic and API calls that were not accounted for in the existing user stories or schedule. When he raised this dependency, the product managers initially preferred to stay with the original plan and timeline. Bart persisted, clearly outlining the user experience risks and technical implications of ignoring the interdependencies, and walked through concrete scenarios to illustrate potential failures. Product ultimately agreed to adjust the roadmap, extend the schedule, and rework the user stories, resulting in a more coherent, maintainable implementation of the feature.

4. “In Bart’s view, what distinguishes a good UI engineer from a great one on a modern front-end stack?”

(Reveals their standards, how they see the role, and whether they align with a high bar for quality and impact.)

Answer: Bart believes a good UI engineer can work effectively within an existing codebase to implement features, fix bugs, and extend components. A great UI engineer goes further by designing clear abstractions for data structures, components, and data flow, so the system remains scalable and maintainable as it grows. They think in terms of architecture—component boundaries, state management, and API contracts—rather than only individual screens. Great UI engineers also collaborate closely with design and product to ensure that technical decisions support a consistent, high-quality user experience over time.

Describe the architecture you’re most proud of designing. How did it evolve over time, and what key factors or performance indicators signaled that told you it was time to refactor or re-platform?

I’m quite proud of the architecture I designed for a major financial data reporting platform that I built while working at Grid Dynamics. The feature set allowed users to choose and apply filters before generating their financial reports. It consisted of a series of drop down menus and a multi-tabbed filter drawer that offered a dynamic set of filters to users to be able to search and select from hundreds of accounts, various date-ranges and custom date ranges, key-word searches, check numbers, lockbox names, etc. Users were also able to save their selections and custom combinations of filters by name and be able to run them again in the future. The tabs displayed recent searches, saved (named) searches, and the set of dynamic filters. I designed the asynchronous data flow from the API, and the UI data-store, which was built with MobX using data maps. In the first generation of this feature, shared components were borrowed from another section of the application (with additional flags and props), including the drawer and the dynamic filter rendering engine, and it used a custom native data store (using specific semantically named observable arrays). However, after further discussions with the product team, I became aware that this feature was not just going to apply to a single report type. It became clear to me that a refactor was necessary in order to accommodate the roadmap of 6 other report types (each of which had their own set of filters, their own APIs etc.).

Rather than continuing to add props and flags to a shared component – making it more complex and error prone, I decided to build out a separate drawer component. The sub-components were still all shared (re-usable / global) components. But the big un-lock was developing observable maps in the MobX datastore. That way, this very complex set of drop down filters, multi-tabbed drawers and all

the related API data flows could be managed in a scalable and re-usable way, simply by using the report-type name as the key to all of the observable arrays.

Give a specific instance where close work with product, design, or data teams changed how you implemented a feature. What trade-offs did you negotiate?

Working with product, design, and data teams, and negotiating trade-offs was a continual part of our process in my work at Grid Dynamics. Somewhat to my surprise considering we were using well-tested components from the Material UI library, issues did often arise in regards to component behavior. But coding customization of the behavior of UI components was a welcome task.

There was one instance however where the available data affected the behavior of the component. The filter drawer had a series of drop-down selections. The first of which was the list of user accounts, followed by a series of other choices such as reporting period or check-number ranges, etc. All of the account data was pulled from API calls that happened before the selection drawer was opened, so there was no delay in displaying the filters. However, in the Figma mock-ups, the product team had introduced a new option. One of the choices was account-status. The user could choose open accounts, closed accounts, or all accounts. It was assumed by the product team that that was a straight forward and static user selection. But I knew from having worked with the accounts API that the back-end was filtering the closed accounts. In order to display the closed accounts we would need to either make a change on the back-end, or place an additional API call to get that data. That led to discussions about how to determine which accounts were closed or not, and it turned out that the accounts data was actually consolidated from multiple origins which each had varying code numbers which determined whether an account was closed, or just dormant. The back-end team was quite backlogged, so they didn't want to take on the task of modifying the accounts API or creating a separate one just for closed accounts, until the next sprint. But the product team didn't want to delay the rollout of this new UI feature, so there was some discussion around changing the accounts API to send all the accounts data and then filtering it on the UI. I warned that trying to rely on filtering the accounts data on the UI was quite risky and could likely cause unexpected behavior and latency issues. Ultimately we agreed to roll out the filter drawer without the ability to choose closed accounts, so that we could get it into the next release, and then add that selection at a future date after the data team and back-end team had worked out their approach for how to handle the account status codes.

When choosing a tech stack for a project, how do you weigh developer velocity, team familiarity and long-term maintenance? Give an example where your initial stack choice aged well or didn't.

Having learned from experience (the hard way) – I think it is important to build out a small prototype project, when choosing a tech stack for a project. This follows the well utilized and prudent strategy of developing an MVP (minimal viable product) – because by building a test project, you can get a sense for how the parts of the stack are working together (or not). It's much easier to iterate and swap out a particular library, plugin, or technology if you're able to isolate it outside of a larger project, or as a feature branch or at the very minimum with a feature flag. Also, having used so many different technologies over the years – I know that long term maintenance is important and so that always weighs heavily on my mind. In terms of team familiarity, I have found that most engineers actually enjoy learning new tech and don't mind diving into something new, as long as the benefits are clearly explained. Also – AI has changed the landscape of development dramatically. With AI incorporated into our workflows and mindsets – it's much easier to learn new tech stacks, new libraries, new platforms, and new languages even. We no longer have to hunt through documentation to find proper

syntax or get up to speed.

Tell us about a recent complex project that you worked on. Why was it complex? What are you personally proud of about that project? Feel free to share a code repository if it was an open sourced project.

I completed a complex project while working at Grid Dynamics that was known as the searchable drop-down filter. While we used the Material UI component library, the product team had envisioned a drop-down menu component that also included a key-word search field to filter the menu items, and they also wanted the ability to select individual list items – and select groups of list items, and collapse others. Material UI offers all of these types of UI components of course, but not all combined into a single encapsulated component that could be re-used in multiple contexts. Working in React, I built a modular component that used a hierarchical (parent/child relationship) data structure. The component displayed the list items in a collapsible tree structure with hover states, selected states, and checkboxes for the selected items. The search field also filtered the list, and previously selected items would move to the top of the list. This project was quite complex because it required maintaining a lot of local state variables, and managing asynchronous data. In other words it required me to get comfortable working with the React rendering lifecycle – understanding when and why the component would re-render, and how to keep those updates in line with user expectations of the behavior. It was also challenging because although this component was originally designed and built for a specific use case (from one product manager on one page)... it quickly became a very popular feature and other product managers and other teams wanted to incorporate it into their pages as well. But in order to accommodate all the various contexts and use-cases, I needed to make all of the functionality (including the asynchronous aspects) have prop-driven control. I was quite proud that my component was quickly adopted by other teams, and became used extensively throughout the app, and I was proud of the fact that it was flexible enough to accommodate varying contexts. For example in some cases it was used inside of a drop-down menu inside a filter-bar, and in other cases it was used as a stand-alone drop-down inside of a drawer. Ultimately I was invited to move it to the global components package. Being able to contribute to the global components codebase was a success for me because it demonstrated that I built the component in a way that was generic, and flexible enough to be re-usable and scalable.

Tell me about an instance where you used AI tools to refactor or migrate a large codebase (e.g., framework upgrade, language version bump). What worked, what didn't?

I have been using AntiGravity and Claude-Code within VS-Code extensively to refactor and migrate various projects. For the most part I find it works amazingly well. One project for instance was built in React with a Node.JS server that served up the React pages. But I used Claude to convert this setup to use NextJS which requires a static build – but serves up static pages faster as a result.

I also recently used Claude Code to refactor my game called LightsOut. It was originally coded in Objective C, and published on the AppStore for iPhone and iPad devices. But I wanted it to also work on Android and on the web, and I wanted to dig in and Learn Flutter & Dart. So I refactored it (by hand the old fashioned way) several years ago. That was a good learning experience and it went fairly smoothly. However, after several years of the project laying dormant, I wanted to add some new features, and get it re-deployed so that I could display in in my portfolio. I found of course that all of the dependencies were out-dated. I was not looking forward to updating, debugging, and-or refactoring them one by one, so I used Claude Code to help me update all of the dependencies and diagnose and resolve any of the issues that I anticipated would arise. It actually worked amazingly well. It updated all of the dependencies – and gave me solid advice when there were decisions to be made in that process. The only big issue that arose really was some layout issues. But it literally only

took about an hour of back-and-forth “vibe-coding” to get those issues resolved. Then I used ChatGPT to hold my hand while I got it deployed to my AWS EC2 instance. I had registered a domain name with a third party hosting service – so ChatGPT helped me diagnose the Nginx and secure hosting issues. While the codebase in this example wasn’t very large, and I am aware that a larger codebase would be more challenging, this process gave me a lot of confidence in the ability to use AI for those types of projects. In my experience of using AI tools, I have found that the main factor that affects quality is remembering to slow down. Sometimes AI works too fast that it can be tempting to try and take on more than you can chew. I enjoy this process quite a bit – and I’ve also found that in general it helps tremendously to know the design patterns and systems of how the code is built – in order to better direct the AI agents, and diagnose problems as they arise together.

he error-state / user-feedback behavior of an input field that I coded for the save-search feature. When users had chosen various filters and wanted to save this filter set for future use, they were asked to give it a name. The name had certain criteria such as needing to be at least 3 characters in length. But naturally, you don’t want to display an error state when the user first starts typing. So if the name was only 2 characters, no error state would display until the user clicked the submit button. However, the design team decided that if the user was editing a previously entered name, and reduced it to only 2 characters, the helper text should appear to warn them they needed to enter more than 2 characters. That same helper text should appear in red

However, there was one instance in particular that was the bane of my existence for quite some time. is the drop-down menus that we used as filters for a list of financial transactions. These menus have a title, and then a list of selections. There was the “Accounts” menu, which had a list of the users banking accounts, and then a “Reporting Period” menu, which had options such as “Year to Date”, and “Prior Year”, etc. The default behavior was that if the user made a selection, that selection would generate a new API call, and refresh the list, and the selections would be displayed as little “chips” underneath the menubar. But the design team wanted unique behavior for the Accounts menu. If the user chose “All Accounts”, the design team wanted the Accounts menu title to change to read “All Accounts”. But if the user chose 4 accounts, the title should just read “Accounts”. Changing the title dynamically was not the standard behavior of the menu as it was coded by the Google Material UI component team. And it wasn’t the expected behavior from the design team for the Reporting Period menu. In that case, the title of “Reporting Period” stayed, regardless of the user-selection.

In most cases I found the back-and-forth discussions around these types of UI behavior issues to be interesting challenges and I welcomed them, and was happy to explain which options were more difficult to engineer versus others.

While at Grid Dynamics, I worked on re-platforming a major U.S. bank’s public website and designed a reusable, searchable multi-select account filter component for the navbar. Originally, the product relied on a Material UI single-select dropdown, which was painful for customers with dozens or hundreds of accounts. I built a React component that accepted a tree data structure, letting users expand and collapse account groups and search by keyword without leaving the navbar. I designed it as a shared component, so other teams could easily integrate it across the application, and it quickly saw

adoption on multiple pages. To handle differing interaction expectations, I exposed flexible configuration props (for example, how to display existing selections during search) without breaking existing implementations. For clients with very large account lists, I collaborated with design and product to switch from a dropdown to a drawer when account counts exceeded a threshold, improving usability and performance for those users.